INTERNSHIP PROJECT REPORT

TASK 3

Heuristic Graph Pathfinding Agent Search Engine

Progree AI Internship Program

By

Name: Shazmeen Irshad

Student ID: B9/2929

1. Introduction:

Pathfinding is an important problem in Artificial Intelligence. It involves finding an efficient route between a starting point and a destination while avoiding obstacles. The purpose of this project is to develop a pathfinding agent that can navigate through grid-based mazes using search algorithms. The agent identifies a path from the starting position to the goal while avoiding blocked cells. This project implements two algorithms: A* Search and Dijkstra's Algorithm. Both algorithms are tested on different maze configurations to evaluate their performance. The project focuses on path length, expanded nodes, runtime, and visualization of the search process.

2. Objectives:

The main objectives of this project are:

- Implement A* Search from scratch using Python.
- Implement Dijkstra's Algorithm from scratch.
- Create grid-based mazes containing obstacles.
- Find paths between starting and goal positions.
- Test algorithms on different maze configurations.
- Measure path steps, runtime, and expanded nodes.
- Calculate expanded node density.
- Visualize the search process.
- Export performance results and visualization logs.

3. Tools and Technologies:

The following technologies were used:

- **Python:** Main programming language.
- **Google Colab:** Development and execution environment.
- **NumPy:** Maze creation and numerical operations.
- **Matplotlib:** Maze and algorithm visualization.
- **Heapq:** Priority queue implementation.
- **Pandas:** Performance results and CSV export.
- **Time:** Runtime measurement.

4. Maze Representation:

The maze is represented as a two-dimensional NumPy array.

Each cell contains one of two values:

- `0` represents a walkable cell.
- `1` represents an obstacle or wall.

The agent moves in four directions: up, down, left, and right.

The initial maze uses the following positions:

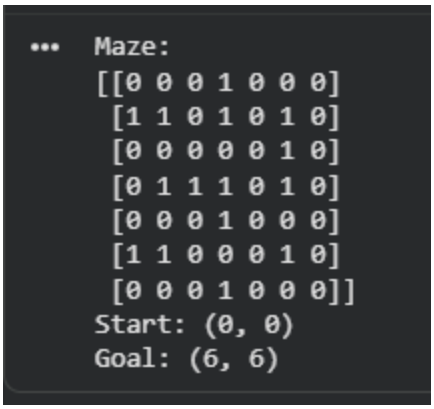
- Start position: `(0, 0)`
- Goal position: `(6, 6)`

The agent cannot move through obstacles or outside the maze boundaries.

Maze Example:

The following grid was used in the implementation:

```
# Create a maze grid
maze = np.array([
    [0, 0, 0, 1, 0, 0, 0],
    [1, 1, 0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0, 0, 0],
    [1, 1, 0, 0, 0, 1, 0],
    [0, 0, 0, 1, 0, 0, 0]
])
```



5. A* Search Algorithm:

A* Search is a heuristic-based pathfinding algorithm. It combines the actual cost of reaching a node with an estimated cost from that node to the goal.

A* uses the following formula:

`f(n) = g(n) + h(n)`

Where:

- `g(n)` represents the cost from the start node to the current node.
- `h(n)` represents the estimated cost from the current node to the goal.
- `f(n)` represents the estimated total path cost.

Manhattan Distance:

This project uses Manhattan Distance as the heuristic function:

`h(n) = |x1 - x2| + |y1 - y2|`

It estimates the distance between two points when movement is restricted to horizontal and vertical directions.

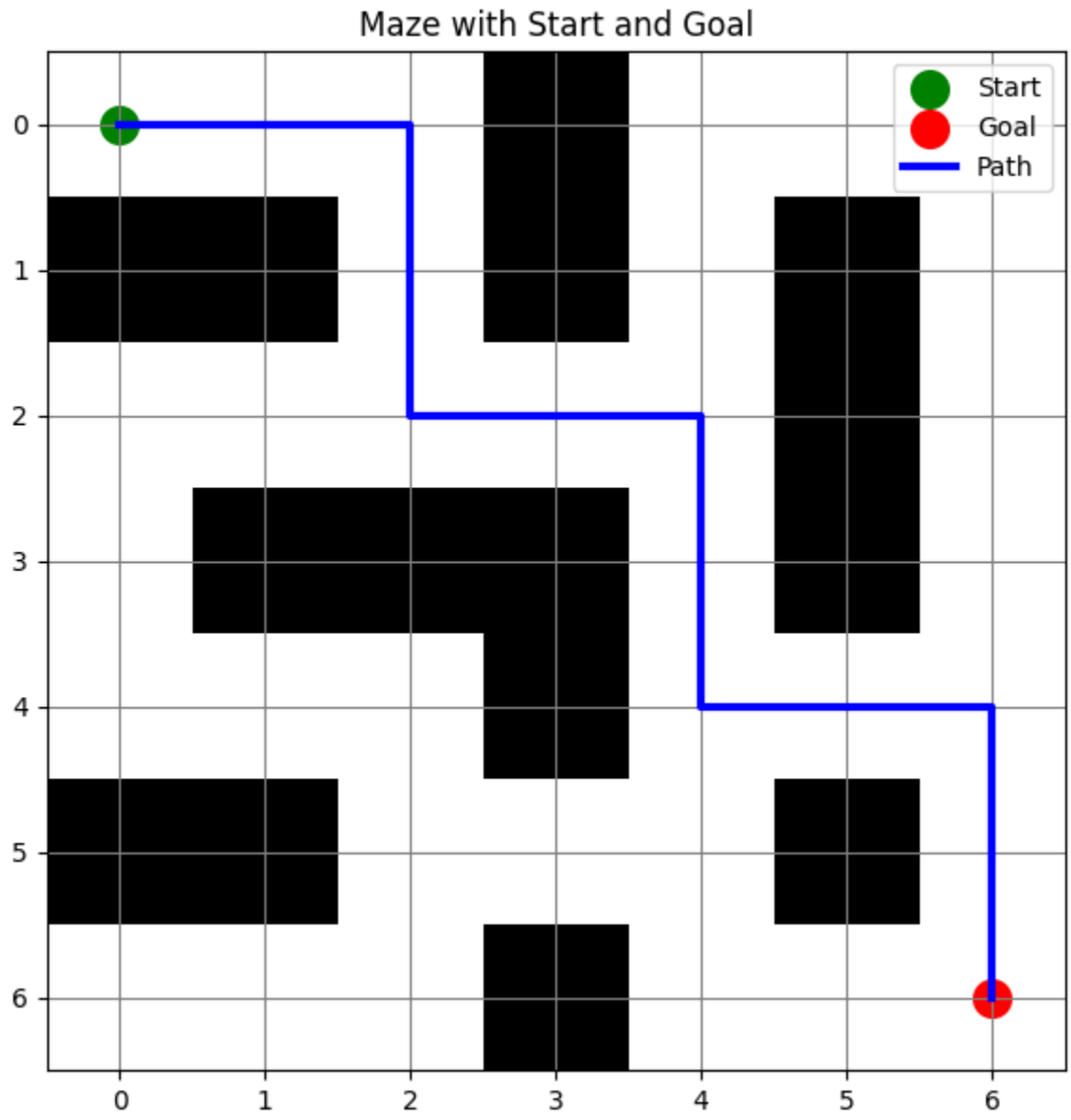
Working Process:

1. Add the starting node to the priority queue.
2. Select the node with the lowest estimated total cost.
3. Check whether the selected node is the goal.
4. Identify valid neighboring cells.

5. Calculate the cost of reaching each neighbor.
6. Update the priority queue when a better path is found.
7. Continue until the goal is reached or no path exists.

A* uses the heuristic to guide its search toward the destination.

```
Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (4, 5), (4, 6), (5, 6), (6, 6)]
Steps: 12
Expanded Nodes: 16
Runtime: 0.0003521442413330078 seconds
```



6. Dijkstra's Algorithm:

Dijkstra's Algorithm is a shortest-path search algorithm that explores nodes according to their accumulated distance from the starting point.

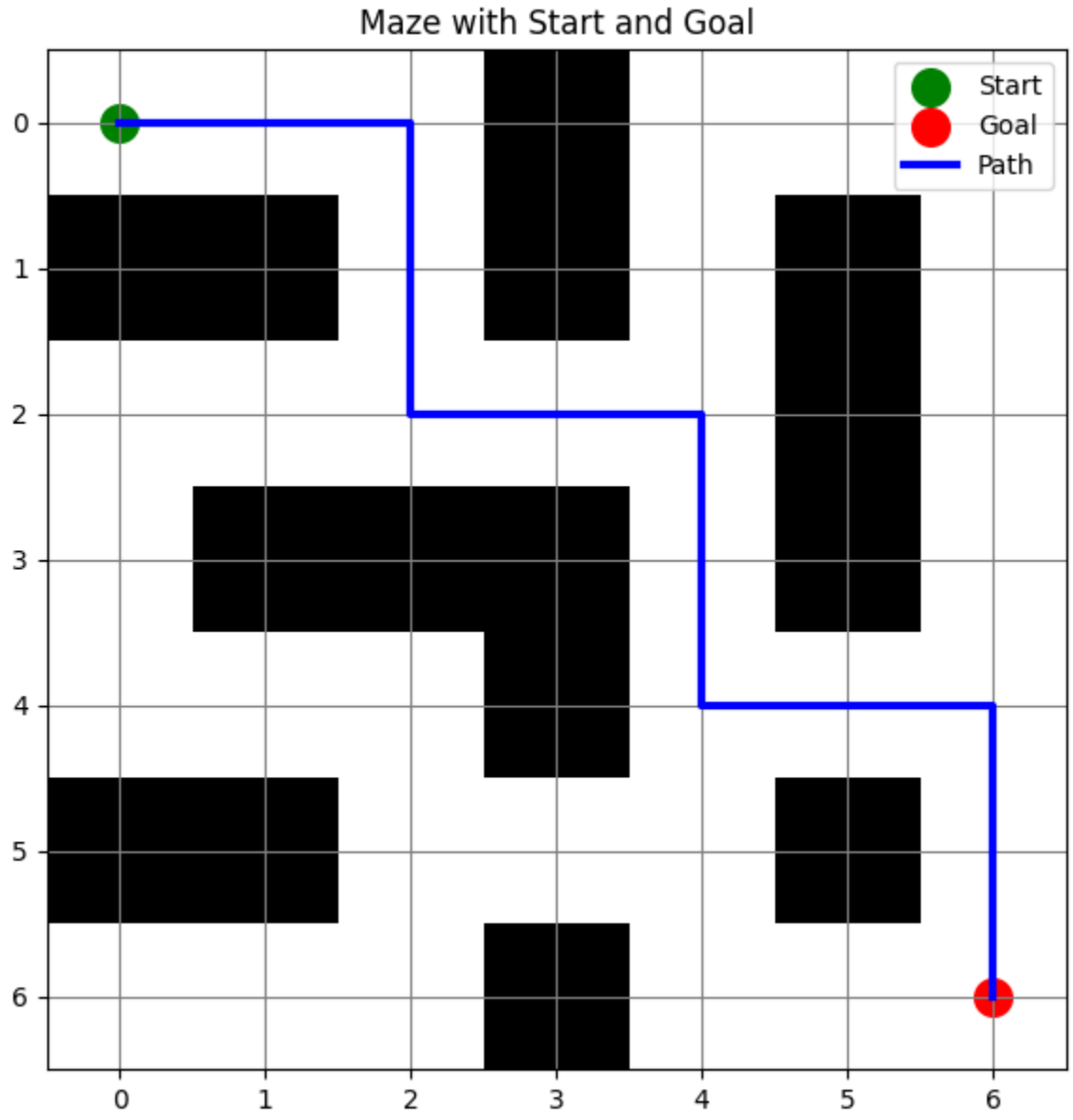
Unlike A*, Dijkstra's Algorithm does not use a heuristic estimate toward the goal.

Working Process:

1. Assign a distance of zero to the starting node.
2. Add the starting node to the priority queue.
3. Select the node with the smallest known distance.
4. Check its neighboring cells.
5. Update distances when a shorter route is found.
6. Continue until the goal is reached or no path exists.

Dijkstra's Algorithm guarantees the shortest path when edge costs are non-negative.

```
Dijkstra Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (4, 5), (4, 6), (5, 6), (6, 6)]
Steps: 12
Expanded Nodes: 32
Runtime: 0.0002696514129638672 seconds
```



7. Implementation:

The project was implemented using Python in Google Colab.

The implementation includes:

- Maze creation.
- Start and goal definition.
- Neighbor identification.
- Manhattan Distance heuristic.
- A* Search implementation.
- Dijkstra's Algorithm implementation.
- Runtime measurement.
- Expanded node tracking.
- Path visualization.
- Performance comparison.
- CSV export.

Both algorithms use priority queues to select the next node for exploration.

8. Experimental Testing:

To evaluate the algorithms, three different maze configurations were tested. Each maze contains a different arrangement of obstacles. Both algorithms were executed using the same start and goal positions.

The following performance metrics were recorded:

Path Steps:

The number of movements required to reach the goal.

Runtime:

The time taken by the algorithm to complete its search. Runtime is measured in seconds and may vary depending on system performance.

Expanded Nodes:

The number of nodes explored by the algorithm during its search.

Expanded Node Density:

Expanded node density represents the percentage of maze cells explored.

It is calculated using:

```
Node Density (%) = (Expanded Nodes / Total Maze Cells) × 100
```

This metric helps evaluate how much of the maze was explored by each algorithm.

	Maze	A* Steps	A* Expanded Nodes	A* Runtime	Dijkstra Steps	Dijkstra Expanded Nodes	Dijkstra Runtime
0	Maze 1	12	16	0.000163	12	32	0.000232
1	Maze 2	12	19	0.000178	12	33	0.000239
2	Maze 3	12	15	0.000129	12	27	0.000165

9. Performance Evaluation:

The algorithms were evaluated based on path steps, expanded nodes, and expanded node density.

The following results were obtained from the notebook:

A* Search Results

Maze	Path Steps	Expanded Nodes	Node Density
Maze 1	12	16	32.65%
Maze 2	12	19	38.78%
Maze 3	12	15	30.61%

Dijkstra's Algorithm Results

Maze	Path Steps	Expanded Nodes	Node Density
Maze 1	12	32	65.31%
Maze 2	12	33	67.35%
Maze 3	12	27	55.10%

10. Results and Comparison:

Metric	A* Search	Dijkstra's Algorithm
Maze 1 Steps	12	12
Maze 2 Steps	12	12
Maze 3 Steps	12	12
Maze 1 Expanded Nodes	16	32
Maze 2 Expanded Nodes	19	33
Maze 3 Expanded Nodes	15	27

Analysis:

A* Search explored fewer nodes in all three mazes.

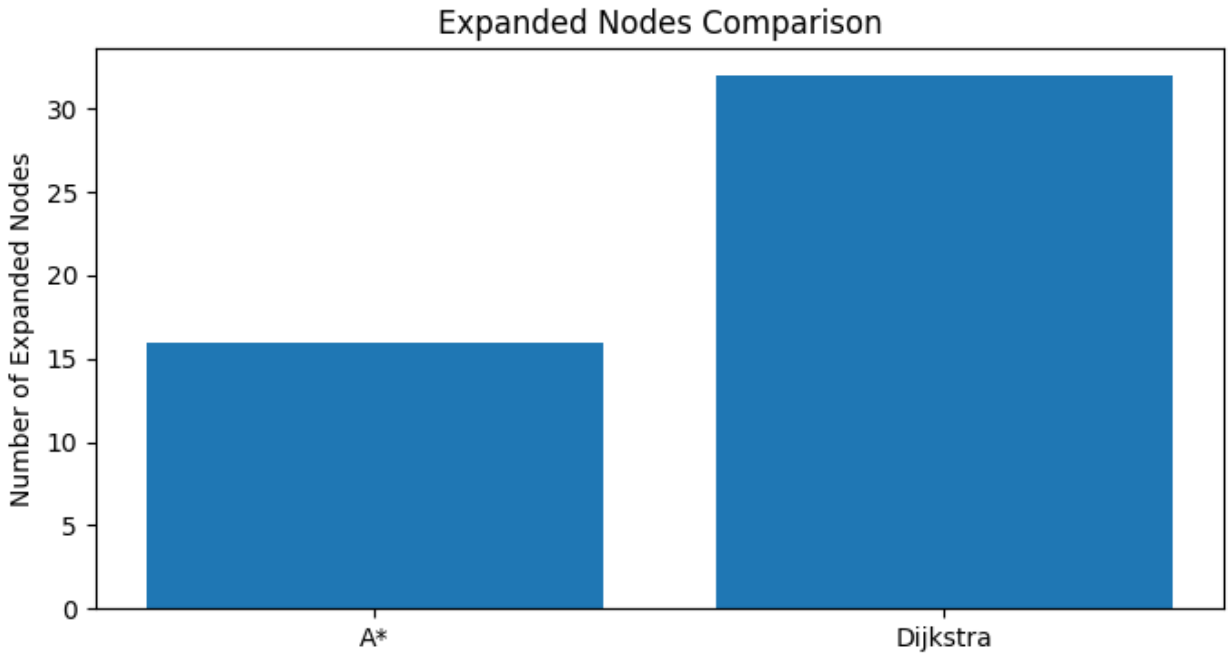
In Maze 1, A* explored 16 nodes, while Dijkstra explored 32 nodes.

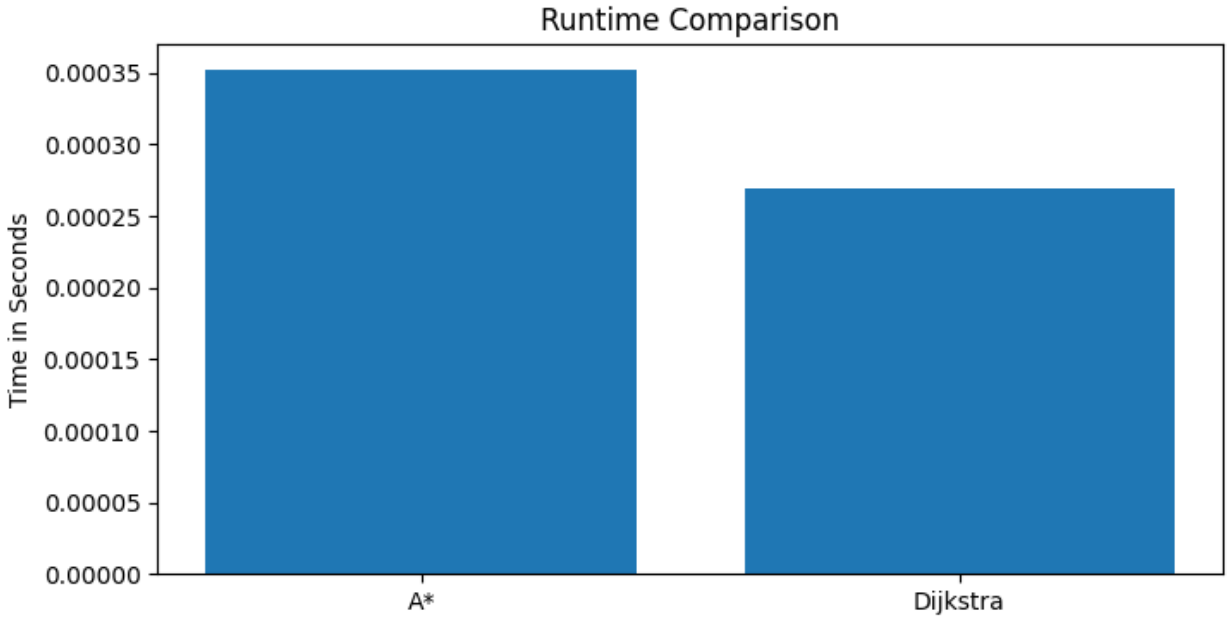
In Maze 2, A* explored 19 nodes, while Dijkstra explored 33 nodes.

In Maze 3, A* explored 15 nodes, while Dijkstra explored 27 nodes.

These results demonstrate that the Manhattan Distance heuristic helped A* focus its search toward the goal.

Both algorithms produced paths with 12 steps in the tested configurations.





11. Visualization and Logs:

Matplotlib was used to visualize the maze and pathfinding process.

The visualization includes:

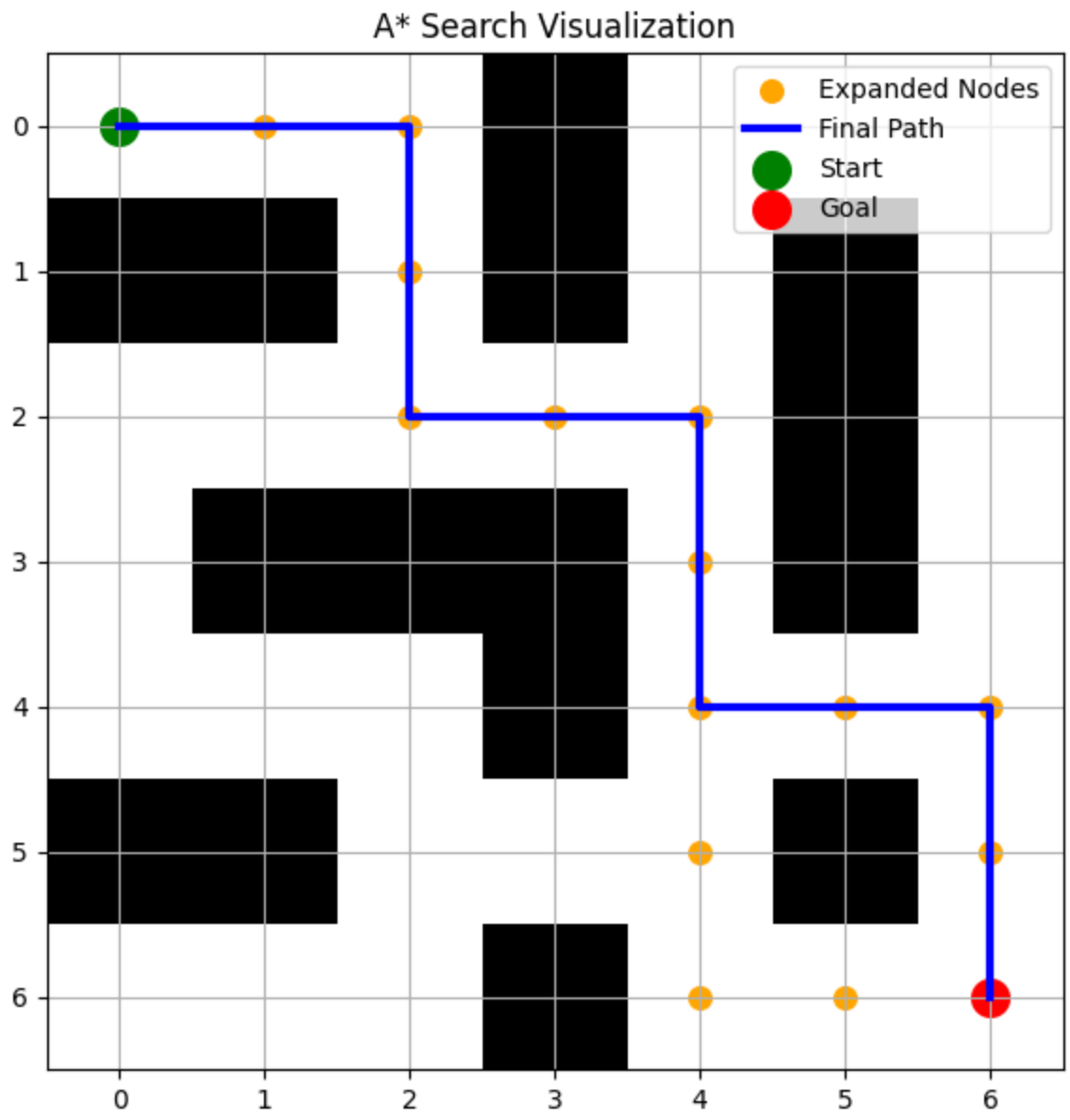
- Maze obstacles.
- Starting position.
- Goal position.
- Expanded nodes.
- Final path.

The project exports the following files:

- **pathfinding_results.csv**
- **astar_search_log.csv**
- **astar_visualization.png**

The CSV files contain performance results and search logs. The visualization image shows the pathfinding process.

These outputs can be used for further analysis and documentation




```
from google.colab import files

files.download("pathfinding_results.csv")
files.download("astar_search_log.csv")
files.download("astar_visualization.png")
```

12. Conclusion:

This project successfully demonstrates the implementation of A* Search and Dijkstra's Algorithm for grid-based pathfinding. Both algorithms were tested on three maze configurations containing obstacles. The results showed that both algorithms found paths with the same number of steps in the tested mazes. However, A* Search explored fewer nodes than Dijkstra's Algorithm. The use of Manhattan Distance helped A* guide its search toward the goal more efficiently. This project demonstrates how pathfinding algorithms can be applied to intelligent agents, robotics, navigation systems, and other optimization problems. The project also provides practical experience in algorithm implementation, performance evaluation, visualization, and data export using Python.